BreakBite · Lab 3 final report

PRM393 · Cloud-Enabled Mobile App (Firebase, ARTEMIS, AI code review) · Team BreakBite (5 members) · repository https://github.com/baonv0817/PRM393_Lab03_Group2 · pinned versions and the map of deliverables: README.md.

> **How to read this report.** Every claim that something works points to a file in the repository (a test, a screenshot in `evidence/` or a document in `docs/`). What could not be shown yet is listed in section 11 instead of being filled in. Screens of P2 were captured on an Android emulator with fake data (no real Firebase, no phone).

1. Project overview and Lab 2 recap

BreakBite is a pre-order, pick-up-at-the-counter food app for students who study on Campus District 9 and have a short break between classes. A **customer app** (S01–S30) lets them order, pay (simulated), follow, edit and review an order; a **seller app** (K01–K15) lets kiosk staff take orders, run the menu and see their shift. Both are one Flutter project; the account role chooses the app. The design is the Lab 2 Figma (`PRM323-Lab2-Group2`, frames with the v2 suffix).

Lab 2 success criterion: finish an order in under 35 seconds with at most 8 taps (docs/analytics-plan.md; the `purchase` event is the one that measures it). **Flows:** F1 quick order, F2 order with a voucher, F3 payment error then cash; plus sign-in, search, export and upload, push notification and the seller side. Main screens built: **44 of 44** (K07 is not in Figma), no placeholder route is left (checked in `lib/routing/routes_p*.dart`).

Member	Area
P1	Ordering flow S03–S08, S21, S22, S25, S29; architecture doc, design traceability, shared theme and widgets
P2	After ordering S09–S12, S20, S23, S24; seller menu K08–K11, K14, K15; PDF receipt export
P3	Firebase: Auth, Firestore, rules, Cloudinary, Crashlytics, Remote Config; S15–S17, S26, S27, Firebase demo area
P4	Seller order handling K01–K06, K12, K13; code review; report, slides, video
P5	S13, S14, S18, S19, S28, S30; FCM; analytics; ARTEMIS tests

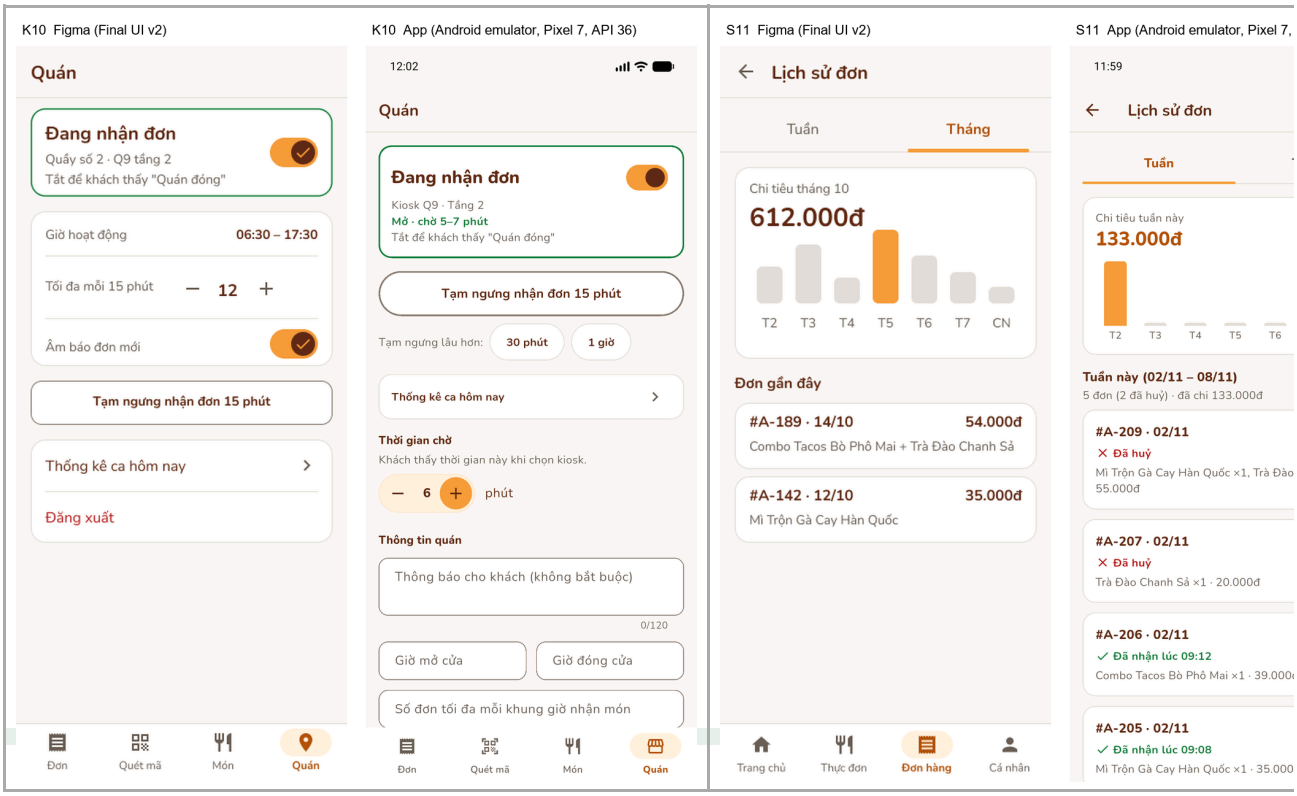
2. Design traceability and deviations

docs/design-traceability.md maps every Lab 2 screen, flow and component to a route, a widget and a data source, with a status. Section 5 of that file records **every place where the app differs from Figma, with the reason** (D01–D33). The main ones:

- **D01, D03:** Google Sign-In replaces the school e-mail + OTP, and a signed-out user only sees S26 (handout requirement).
- **D12, D13:** the add-to-cart stepper and the floating cart button replace the + icon and the cart icon of S02 (the cart must be reachable and correctable).
- **D18:** S25 offers the kiosk's break slots (with a capacity) instead of a typed timetable, because the data model books slots.
- **D23, D24:** K14/K15 have no photo box and no "Xoá món": the final assignment drops Cloud Storage and the rules forbid deleting menu items.

- **D21, D22, D25–D27:** the screens of P2 add what the data model already had (wait time, notice, hours and capacity in K10; the line "Đang xử lý" in K11; weekly and monthly groups in S11; a timeline, review and receipt in S12).

The column "Figma updated" is still **Not yet** for these rows; updating Figma is task T83 (section 11). Two examples of Figma (left) and the app (right) from [evidence/figma/]:



K10 "Quán" (D21) and S11 "Lịch sử đơn" (D25). Fourteen such pairs for the screens of P2 are in `evidence/figma/`, thirteen more for P1.

3. Architecture and MVVM

MVVM with Provider and go_router, following the official Flutter architecture guide: a **UI layer** (screens and ViewModels grouped by feature, `lib/ui/<feature>/`) above a **data layer** (repositories and services, `lib/data/`) above pure models (`lib/domain/`). Dependencies only point down. [docs/architecture.md] explains why the folders differ from the handout's suggestion (five people working in parallel; the final assignment needs a shared package).

Layer	Responsibility	Knows about
Screen `ui/*/widgets`	Draws the ViewModel's `ViewState`, forwards taps	Its ViewModel, `ui/core`, routes
ViewModel `ui/*/view_models`	Screen state, validation, calls to repositories and services, `Command` for actions	Interfaces and models only
Repository `data/repositories`	Business data in Firestore → models, queries, transactions	Models, Firestore (in its implementation)
Service `data/services`	Wraps one SDK (Auth, FCM, Analytics, Crashlytics, Remote Config, Cloudinary)	The SDK (in its implementation)
Model `domain/models`	Immutable data and pure rules	Nothing

Enforced by a test: `test/architecture_test.dart` fails if a screen, a ViewModel, a route or a

model imports an SDK, if a screen holds a `Color`, `Colors` or `TextStyle` literal, or if a ViewModel does not use `SafeChangeNotifier`. Fake data lives in `dev/`, outside `lib/`, so the submitted build has no mock data. **MVVM check across the repo (T72):** every widget file was read against the rules; three pieces of logic were moved out of `build()` into ViewModels (S11, S12, K11) and two cases were kept with a reason (section 3 of the architecture document).

4. Design system in code

Tokens live in one place, `lib/ui/core/theme/`: `app_colors.dart` (colors from the Figma v2 frames, with contrast tests for WCAG), `app_dimens.dart` (spacing, radius, minimum tap target 48 dp) and `app_theme.dart` (the Material theme built from them). Components are one widget each in `lib/ui/core/widgets/` (`AppButton`, `AppChip`, `AppTopBar`, `QuantityStepper`, `BottomActionButton`, `ViewError`/`ViewEmpty`/`LoadingView`, `StatusBadge`…); every screen uses them, and the architecture test forbids colour or text-style literals in screens. The widget catalog `dev/widget_catalog.dart` shows every component and variant and is tested at 360 and 412 dp with text at 1.0 and 1.3: no overflow, every tap target at least 48 dp, every one labelled. Every state a screen can be in (loading, empty, error with retry) is a shared widget, not a copy.

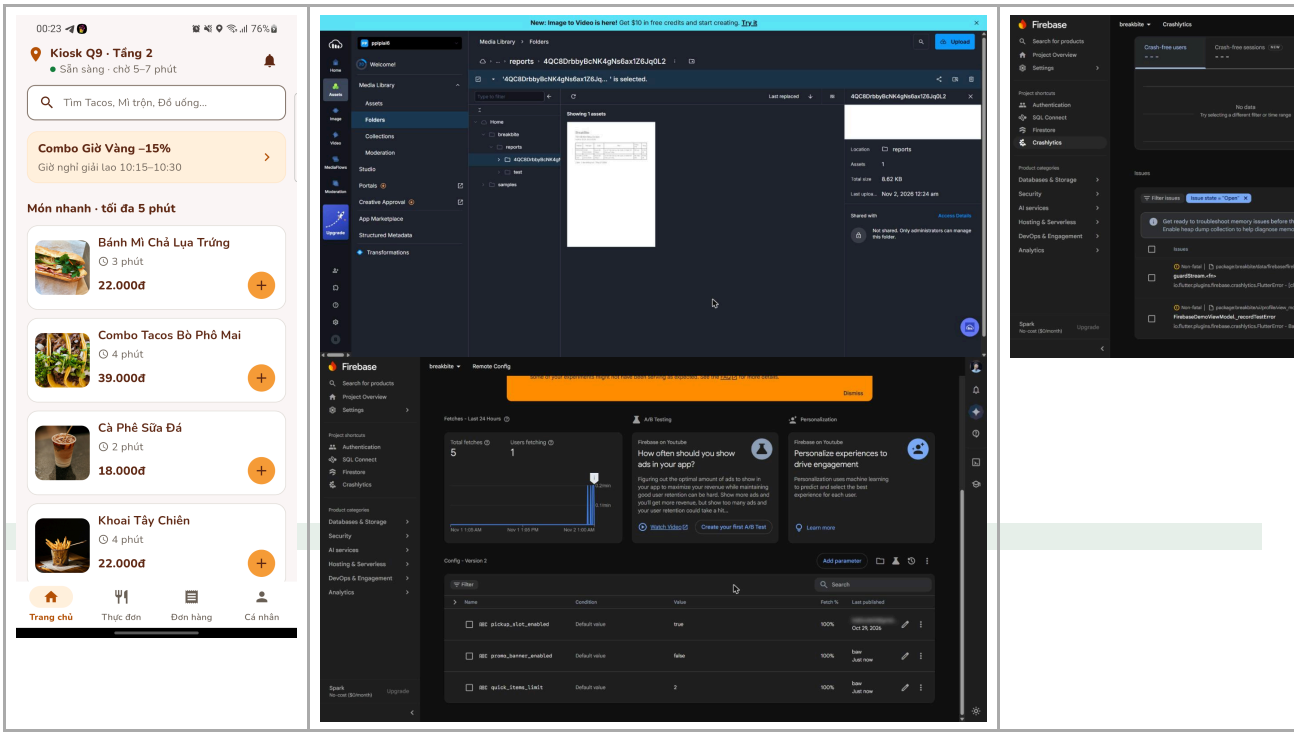
5. Firebase design

Project `breakbite-1315b`, Spark plan. Screens never call an SDK: each service is an interface in `lib/data/services/` with a Firebase implementation, and every failure comes back as a typed `AppException` with a Vietnamese message (docs/firebase-design.md has the full table per service).

Service	What the app does	Failure handling
Authentication	Google Sign-In (S26); first sign-in creates `users/{uid}` with role `user`; the role (set with `set-role.mjs`) routes to the customer or the seller app; sign-out clears local data	Cancelled sign-in, no network and "no token" each show their own message; the router waits on a splash screen until the sign-in state is known
Firestore + rules	Menu, cart, orders (placed, edited and cancelled in one transaction), incidents, reviews, notifications; deny by default, access by role, staff only at their own kiosk; 88 rules tests and an integration test against the real rules	A write that depends on current data runs in a transaction, so the user sees a clear reason (for example "Đơn này vừa được xử lý ở nơi khác") instead of a rules error
Files (Cloudinary)	The order receipt (S12) and the report in the demo area are PDFs uploaded with a progress bar to `breakbite/reports/{uid}/`; the URL and metadata are kept in `reports` (owner-only rules)	Not configured, larger than 5 MB, no network, timeout and "PDF cannot be made" each have a message; the retry button stays
Cloud Messaging	Permission on Android 13+, token per user in `users/{uid}.fcmTokens`, messages saved to the in-app list (S19) in foreground, background and terminated states; tapping opens the order	A refused permission or a failed token write never blocks sign-in or sign-out
Analytics	11 events in the plan, all logged in code; `setUserId` after sign-in, no personal data	Logging never throws

Crashlytics	Uncaught errors reported; handled errors from real error paths recorded as non-fatal with the keys `role` and `kiosk_id`	Reporting never throws
Remote Config	`quick_items_limit`, `promo_banner_enabled`, `pickup_slot_enabled`; the app listens for updates, so Home changes without a rebuild	Offline: last values, or the in-app defaults on a first launch

Why Cloudinary: Cloud Storage for Firebase needs the Blaze plan (a payment card). The upload goes through the same `StorageService` interface with an unsigned preset, so the app holds no API secret; this is a deviation from the handout (listed in the README, approval of the lecturer still to be attached, section 11).



Signed-in app (`evidence/auth`), the uploaded PDF in the Cloudinary library (`evidence/storage`), the Remote Config console (`evidence/remote-config`) and a non-fatal error in Crashlytics (`evidence/crashlytics`), all from project `breakbite-1315b`.

6. Analytics plan and evidence

docs/analytics-plan.md was written before the code: 11 events covering sign-in (`login`), sign-out (`logout`), search, opening an item (`view_menu_item`), ordering (`add_to_cart`, `reorder`, `begin_checkout`, `purchase`, `payment_failed`), the seller steps (`order_accepted`, `order_delivered`) and the export (`export_report`, with `source: order_detail` for the receipt of S12). Each event answers a product question and maps to a Lab 2 step; the one that tests the success criterion is `purchase` (≤ 8 taps, < 35 seconds). Names come from `AnalyticsEvents`, parameters are numbers and short strings, never a name or an e-mail. All 11 are logged in code and covered by ViewModel tests. **The DebugView screenshots are not collected yet** (section 11).

7. ARTEMIS test plan, results and exploratory session

docs/test-plan.md has 31 test cases (TC-01 to TC-31) covering sign-in and sign-out, F1–F3, search, export and upload, push, the seller flow, the seller menu, a sold-out item and a closed

kiosk, with happy paths, error paths and native interactions (airplane mode, notification). e2e_test/ has one pytest function per case; each sends one instruction and **asserts on the value the agent reports back**, not only on `succeeded`; `run_repeat.py` runs the suite N times and writes the pass-rate table. P2's cases are TC-18 to TC-29 (`test_order_manage.py`, `test_seller_menu_shop.py`).

Results of ARTEMIS runs: none yet. ARTEMIS needs the ARTEMIS server, a model API key and a device (T02); the tests are written and the "Result" column of the test plan is still "Not run". Nothing was filled in without a run. What **was** run: 749 unit, ViewModel, widget and full-app flow tests (`flutter test`, no device), 88 Firestore rules tests (`firebase/tests`) and 15 integration tests of the real Firebase code against the Auth and Firestore emulators on an Android emulator (`integration_test/firebase_emulator_test.dart`). `flutter analyze` has no issue.

Exploratory session

10/11/2026, about 10 minutes, by P2 with Claude Code on an Android emulator with fake data, driven with `adb`; **not** the ARTEMIS Pro-profile session of the handout (P5 repeats it on a phone). Charter: text at 200 %, fast taps, odd search input, the receipt export. Five problems, all low severity (docs/test-plan.md section 4):

Found	Handled
Orders tab label "Đang xử lý (2)" fades at the bracket at 200 % text (S08)	Open
Kiosk status line cut with "…" on Home and Menu at 200 % text, the wait time is lost	Open
Empty search says "xem các món bán chạy" but offers only search suggestions	Open
Badge "Đã nhận đơn" above "Chờ kiosk nhận đơn", and "Đã nhận" for a picked-up order, read as contradictory	Open, wording from Figma
K10 time-field labels cut ("Giờ mở c…") at 200 % text	Fixed: fields stack at ≥ 1.5× text, widget test added

No problem found: 14 fast taps on "+" (quantity 14, total 546.000đ correct), two quick taps on the receipt export (one receipt), the seller tabs at 200 % text (no overflow).

8. AI code review findings and decisions

Tool: Claude Code, skill `/code-review`, run on the repositories and the screens of P2; the unchanged output is in evidence/code-review/claude-code-review.md and the full table in docs/code-review.md. Each finding was opened in the code before it was decided, because the tool can be wrong (for CR-10 its text said the description is untrimmed, but the repository trims it).

CR-01 · security · `firestore_order_repository.dart:52` · fix: —

Finding: The order total is built from client prices, never read from `menu_items`.

Decision: Accepted, not fixed: needs menu reads in the order transaction and a rules change on the ordering path of P1/P3; recorded as a known limitation.

CR-02 · bug · `firestore_menu_repository.dart:91` · fix: d6d8113

Finding: A new item with an existing name overwrites that item.

Decision: Accepted: silent data loss between two staff members.

CR-03 · bug · `firestore_incident_repository.dart:98` · fix: d1291a6

Finding: Take/answer an incident checked a stale copy.

Decision: Accepted: rules let staff update a resolved incident, so a second answer overwrote the first; now a transaction.

CR-04 · performance · `firestore_notification_repository.dart:55` · fix: 9243bf3

Finding: "Mark all read" in one write batch.

Decision: Accepted: Firestore caps a batch at 500 writes; now batches of 400, tested with 450.

CR-05 · bug · `order_history_view_model.dart:98` · fix: 2980559

Finding: "Xem đơn cũ hơn" could add nothing while older orders exist.

Decision: Accepted: the first load already skips such pages; test fails before, passes after.

CR-06 · bug · `seller_menu_view_model.dart:41` · fix: ba8dc33

Finding: First "Có lại lúc" option ignored seconds.

Decision: Accepted: 10:15:20 offered 10:30, 20 s under the promised 15 minutes.

CR-07 · bug · `firestore_order_repository.dart:275` · fix: —

Finding: Slot release fails if the slot document is missing.

Decision: Rejected: the rules allow no slot delete and keep `booked` ≥ 0; no script deletes slots.

CR-08 · maintainability · `shop_view_model.dart:134` · fix: —

Finding: `_apply` returns ok when busy.

Decision: Rejected: the screen disables every control while a write runs, and a test covers the guard.

CR-09 · performance · `shift_stats_view_model.dart:116` · fix: —

Finding: K11 reads all kiosks to get one opening time.

Decision: Rejected: 2 kiosks, 2 reads per refresh; a new repository method costs more than it saves.

CR-10 · test gap · `incident_view_model.dart:128` · fix: —

Finding: Submit does not check status or an earlier report.

Decision: Rejected: partly wrong; the button is disabled while running and the rules refuse a second report.

Five fixes, each in its own commit that names the finding; the new tests fail on the old code and pass on the new (CR-05 was run both ways). CR-02, CR-03 and CR-04 are also covered by the emulator integration test.

9. Challenges encountered

- **Cloud Storage is not free.** It needs the Blaze plan, so files go through Cloudinary behind the same `StorageService` interface; the lecturer's agreement is still to be attached (section 11).

- **Five people on 44 screens.** One folder per feature and one route file per owner kept merges small; a few shared files (theme, widgets, router) needed rules in `CONTRIBUTING.md`.
- **Rules and UI must agree.** A check done only in the UI is not a security rule: the incident rules let staff update a resolved report, and an order's prices are not checked against the menu (CR-01, CR-03). Testing the real rules on the emulator found both kinds of gap that fake data hides.
- **Stale data on screen.** Two staff members can hold different copies of the same document; the fixes read the document again inside a transaction.
- **Evidence needs accounts.** Real-Firebase screenshots need a staff role (set by P3 with `set-role.mjs`) and SHA-1 keys of every machine; until then the screens were captured with fake data.

10. Lessons learned

- Write the traceability and the test plan **before** the screens: every deviation found later cost a table row and a Figma change.
- Put logic in ViewModels from the start: moving it out of `build()` afterwards (T72) was three small changes only because the architecture test already blocked the worst cases.
- Test against the real rules early. The emulator test caught assumptions that the fakes could not (for example that a customer may report an order that is not delivered yet: the rules allow it, the UI does not offer it).
- An AI review is a list of questions, not of facts: four of ten findings were rejected after reading the code and the rules, and one claim in an accepted-looking finding was wrong.
- Do not mark a test as passed without a run: the ARTEMIS "Result" column stays "Not run" until the server and a device are available.

11. Evidence still to be collected

Item	Why it is missing	Who
ARTEMIS results (3 runs per test, pass-rate table), one run for the video	Needs the ARTEMIS server, a model API key and a device (T02)	P5
Analytics DebugView screenshots, one per event	Not collected yet	P5
FCM screenshots in the three states, S19	Not collected yet	P5
Real-Firebase run of the seller screens K08–K15 with a staff account	Needs the staff role (`set-role.mjs`) and each machine's SHA-1	P3, P2
Receipt upload from S12 on a device and in the Cloudinary library (T43)	The demo-area upload is evidenced (section 5); the S12 receipt is not	P2
Written confirmation from the lecturer for Cloudinary, a team of 5 and D01 (T11)	Not obtained yet	P4
Figma updated for the deviations (D01–D33), the demo video, the submission checklist (T82, T83)	Done at the end by the team	P1, P4, all
Known limitation CR-01 (order prices are not checked against the menu on the server)	Needs a rules change on the ordering path	P3

12. AI tools used

Tool	Purpose
Claude Code (Claude Sonnet 5.5)	Writing and testing the code of P2, the ARTEMIS cases TC-18 to TC-29, the evidence screenshots of P2, the review fixes, this report and the slides (generated from the repository's own documents by `scripts/build_report.py`)
Claude Code, skill `/code-review`	The AI code review of section 8
Other members	To be added by each member in the README and in docs/code-review.md